

Table des matières

Labo 07 OLAP — Sous-requêtes, corrélation et ensembles sur l’entrepôt 1

 Le terrain, en une minute 1

 Partie 1 — Les trois formes, et la corrélation 2

 Partie 2 — Les questions que le GROUP BY ne sait pas poser 3

 Partie 3 — Ce que le contraste vous a montré 7

 Et ensuite ?..... 7

Labo 07 OLAP — Sous-requêtes, corrélation et ensembles sur l’entrepôt

Laboratoire du Cours 07, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

Ce laboratoire se fait dans la ligne de commande DuckDB, sur l’entrepôt monté au Cours 00. Comptez trois quarts d’heure. Tapez les requêtes, ne les copiez pas : c’est en tapant qu’on remarque ce qu’on n’a pas compris.

Les deux voies — Ce labo a un jumeau, Labo_07_OLTP.md, qui pose les mêmes questions sur la base transactionnelle dans PostgreSQL. Faites celui-ci d’abord si vous venez du monde analytique, ou l’autre d’abord si vous venez du développement applicatif. Faites les deux dans tous les cas : c’est le contraste qui enseigne.

Le terrain, en une minute

Quatre tables suffisent pour tout ce module.

Table	Ce qu’elle contient	Colonnes utiles ici
FactInternetSales	une ligne par ligne de commande en ligne	ProductKey, CustomerKey, SalesOrderNumber, OrderQuantity, SalesAmount, OrderDateKey
DimProduct	le catalogue, en dimension à évolution lente	ProductKey, EnglishProductName, ListPrice, ProductSubcategoryKey
DimProductSubcategory	les sous-catégories	ProductSubcategoryKey, EnglishProductSubcategoryName, ProductCategoryKey
DimProductCategory	les quatre catégories	ProductCategoryKey, EnglishProductCategoryName

Posez le contexte une fois pour toutes :

```
USE dbo;
```

Une particularité de l'entrepôt qu'il faut connaître avant de commencer : DimProduct compte **606 lignes pour 504 produits**, parce que certains produits y ont plusieurs versions historiques.

Attention à ce que cela fait et à ce que cela ne fait pas. Une jointure du fait vers la dimension sur ProductKey ne double rien : cette clé désigne une version précise, et chaque ligne de vente n'en rencontre qu'une. Le vrai effet est ailleurs, et il est plus sournois. Une anti-jointure qui cherche les produits jamais vendus travaille elle aussi au grain de la version : elle liste comme « jamais vendues » 19 lignes qui sont l'ancienne version d'un produit bel et bien vendu sous sa version courante, soit 11 noms de produits. Un rapport qui titrerait « onze produits n'ont jamais trouvé preneur » serait faux, et aucune requête ne l'aurait signalé.

La leçon est celle du grain, encore : COUNT(DISTINCT ...) sur la clé métier plutôt que sur la clé de version est ce qui ramène le compte à la question posée. Le Cours 09 traite le sujet en profondeur ; ici, on s'en sert pour comprendre pourquoi ce DISTINCT n'est pas une précaution de style.

Partie 1 — Les trois formes, et la corrélation

1.1 La sous-requête scalaire

Théorie : Theorie_07.md, section 2.

Trouvez les produits dont le prix de liste dépasse la moyenne du catalogue.

```
SELECT p.EnglishProductName, p.ListPrice
FROM   DimProduct AS p
WHERE  p.ListPrice > (SELECT AVG(ListPrice) FROM DimProduct)
ORDER BY p.ListPrice DESC;
```

Attendu : une centaine de lignes, les vélos en tête.

Maintenant la même chose, mais avec l'écart affiché. Comparez les deux écritures :

-- Version sous-requête : La table est relue pour calculer la moyenne.

```
SELECT p.EnglishProductName,
       p.ListPrice,
       p.ListPrice - (SELECT AVG(ListPrice) FROM DimProduct) AS ecart
FROM   DimProduct AS p;
```

-- Version fenêtre du Cours 04 : un seul passage.

```
SELECT p.EnglishProductName,
       p.ListPrice,
```

```

    p.ListPrice - AVG(p.ListPrice) OVER () AS ecart
FROM DimProduct AS p;

```

Le résultat est identique. Ouvrez le *Query Profile* des deux exécutions (le bouton en haut à droite d'un résultat, onglet *Query Profile*) et comparez le nombre de parcours de *DimProduct*. C'est le genre de vérification que le Cours 10 systématisera.

1.2 La sous-requête corrélée

Théorie : Theorie_07.md, section 3.

Pour chaque produit, comptez ses lignes de vente.

```

SELECT p.EnglishProductName,
       (SELECT COUNT(*)
        FROM FactInternetSales AS f
        WHERE f.ProductKey = p.ProductKey) AS nb_lignes
FROM DimProduct AS p
ORDER BY nb_lignes DESC
LIMIT 10;

```

Attendu : une dizaine de lignes, les accessoires en tête (ils se vendent en grand nombre, pas en grande valeur).

Reprenez la requête en la préfixant de *EXPLAIN*. Vous n'y trouverez pas de boucle : le plan affiche un *LEFT_DELIM_JOIN*, un *HASH_JOIN* et un *HASH_GROUP_BY*. DuckDB a décorrélé la requête et l'a transformée en jointure avec agrégation, ce que la section 3 de la théorie annonce. Le mot du plan, *delim join*, est le nom technique de cette transformation.

1.3 La sous-requête de table

Théorie : Theorie_07.md, section 2.

```

SELECT s.ProductKey, s.qte_totale
FROM (SELECT ProductKey, SUM(OrderQuantity) AS qte_totale
      FROM FactInternetSales
      GROUP BY ProductKey) AS s
WHERE s.qte_totale > 1000
ORDER BY s.qte_totale DESC;

```

Réécrivez-la en CTE, comme au Cours 05, et gardez la version que vous trouvez la plus lisible. La réponse attendue est la CTE : elle se lit de haut en bas.

Partie 2 — Les questions que le GROUP BY ne sait pas poser

2.1 Les produits jamais vendus, par anti-jointure

Théorie : Theorie_07.md, section 4.

```
SELECT p.EnglishProductName, p.ListPrice
FROM   DimProduct AS p
WHERE  NOT EXISTS (SELECT 1
                   FROM    FactInternetSales AS f
                   WHERE   f.ProductKey = p.ProductKey)
ORDER BY p.EnglishProductName;
```

Attendu : plusieurs centaines de lignes. Beaucoup de produits de l'entrepôt ne sont vendus qu'aux revendeurs, donc absents de FactInternetSales.

Écrivez maintenant la même question avec NOT IN, puis avec un LEFT JOIN :

```
-- Avec NOT IN.
SELECT COUNT(*)
FROM   DimProduct AS p
WHERE  p.ProductKey NOT IN (SELECT f.ProductKey FROM FactInternetSales AS f);

-- Avec LEFT JOIN et test d'absence.
SELECT COUNT(*)
FROM   DimProduct AS p
LEFT   JOIN FactInternetSales AS f ON f.ProductKey = p.ProductKey
WHERE  f.ProductKey IS NULL;
```

Les trois comptes doivent coïncider ici, parce que FactInternetSales.ProductKey ne contient pas de NULL. Gardez ce résultat en tête : la partie 3 vous fera fabriquer le cas où ils divergent.

2.2 Le piège NOT IN, provoqué exprès

Théorie : Theorie_07.md, section 5.

Fabriquez une liste qui contient un NULL et observez.

```
-- Combien de produits ne sont pas dans la liste (1, 2) ?
SELECT COUNT(*) AS avec_valeurs
FROM   DimProduct AS p
WHERE  p.ProductKey NOT IN (1, 2);

-- La même question, avec un NULL ajouté à la liste.
SELECT COUNT(*) AS avec_null
FROM   DimProduct AS p
WHERE  p.ProductKey NOT IN (1, 2, NULL);
```

Attendu : la première renvoie 604, la seconde renvoie **0**. Aucune erreur, aucun avertissement. C'est le résultat le plus important du laboratoire. Refaites-le avec NOT EXISTS sur une sous-requête qui renvoie ces mêmes valeurs, et constatez que le compte reste juste.

2.3 Les clients d'une catégorie mais pas d'une autre

Théorie : Theorie_07.md, section 6.

On utilise ici les opérateurs d'ensembles. Commencez par la vue de travail :

```
WITH ventes_par_categorie AS (  
    SELECT f.CustomerKey, c.EnglishProductCategoryName AS categorie  
    FROM FactInternetSales AS f  
    JOIN DimProduct AS p ON p.ProductKey = f.ProductKey  
    JOIN DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey  
    JOIN DimProductCategory AS c ON c.ProductCategoryKey = sc.ProductCategoryKey  
)  
SELECT CustomerKey FROM ventes_par_categorie WHERE categorie = 'Bikes'  
EXCEPT  
SELECT CustomerKey FROM ventes_par_categorie WHERE categorie = 'Accessories';
```

Attendu : quelques milliers de clients. Ce sont ceux qui ont acheté un vélo sans jamais prendre d'accessoire, une population que le service commercial voudra sûrement rappeler.

Remplacez EXCEPT par INTERSECT pour obtenir les clients des deux catégories, puis par UNION ALL et comparez le nombre de lignes avec UNION. L'écart entre les deux derniers est exactement le nombre de doublons que UNION a supprimés, à un coût que le *Query Profile* montre.

2.4 Le top 3 par sous-catégorie, en trois écritures

Théorie : Theorie_07.md, sections 2 et 7.

D'abord avec LATERAL :

```
SELECT sc.EnglishProductSubcategoryName AS sous_categorie,  
       top.EnglishProductName,  
       top.ListPrice  
FROM DimProductSubcategory AS sc,  
     LATERAL (SELECT p.EnglishProductName, p.ListPrice  
              FROM DimProduct AS p  
              WHERE p.ProductSubcategoryKey = sc.ProductSubcategoryKey  
              ORDER BY p.ListPrice DESC  
              LIMIT 3) AS top  
ORDER BY sous_categorie, top.ListPrice DESC;
```

Puis avec QUALIFY, la forme la plus courte :

```

SELECT sc.EnglishProductSubcategoryName AS sous_categorie,
       p.EnglishProductName,
       p.ListPrice
FROM   DimProduct AS p
JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
QUALIFY ROW_NUMBER() OVER (PARTITION BY sc.ProductSubcategoryKey
                           ORDER BY p.ListPrice DESC) <= 3
ORDER BY sous_categorie, p.ListPrice DESC;

```

Puis avec une fenêtre et une CTE, la forme portable :

```

WITH classe AS (
    SELECT sc.EnglishProductSubcategoryName AS sous_categorie,
           p.EnglishProductName,
           p.ListPrice,
           ROW_NUMBER() OVER (PARTITION BY sc.ProductSubcategoryKey
                               ORDER BY p.ListPrice DESC) AS rang
    FROM   DimProduct AS p
    JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
)
SELECT sous_categorie, EnglishProductName, ListPrice
FROM   classe
WHERE  rang <= 3
ORDER BY sous_categorie, ListPrice DESC;

```

Les trois donnent le même résultat. Notez que LATERAL conserve les sous-catégories sans produit seulement si on écrit LEFT JOIN LATERAL ... ON true ; avec la virgule, elles disparaissent. Les deux autres écritures les perdent aussi, puisqu'elles partent de DimProduct.

2.5 Une anti-jointure de contrôle qualité

Théorie : Theorie_07.md, section 4.

Voici l'usage professionnel de l'anti-jointure : chercher les lignes de fait dont la dimension est absente. Ce sont les erreurs d'alimentation les plus coûteuses, parce qu'elles font disparaître du chiffre d'affaires d'un rapport sans laisser de trace.

```

SELECT COUNT(*) AS lignes_orphelines
FROM   FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM DimProduct AS p WHERE p.ProductKey = f.ProductKey);

```

Attendu : zéro. AdventureWorks DW est un jeu de démonstration propre sur ce point. Un zéro n'est pas une déception : c'est le résultat qu'on espère, et c'est le contrôle qu'on met en place **avant** d'en avoir besoin. Refaites-le sur CustomerKey et sur OrderDateKey.

Contrôle de cohérence — Gardez cette requête. C’est le geste qu’un ingénieur analytique répète partout : « ai-je des transactions qui référencent un instrument inconnu ? », « des ventes dont le magasin n’existe plus dans le référentiel ? ». La forme ne change jamais, seules les tables changent.

Partie 3 — Ce que le contraste vous a montré

Théorie : Theorie_07.md, section 8.

Vous verrez la colonne PostgreSQL se remplir en faisant Labo_07_OLTP.md. Notez déjà ce que l’entrepôt vous a appris.

Observation	Sur l’entrepôt
Le top N par groupe	QUALIFY en fait la forme la plus courte des deux terrains, en une seule requête sans CTE
LATERAL	s’écrit avec une virgule, sans le mot-clé JOIN, mais perd alors les lignes de la table de gauche sans correspondance
EXCEPT	pas de version ALL. MINUS en est un synonyme
Le piège NOT IN	se démontre sur une liste littérale, faute de colonne nullable évidente dans le fait
La corrélation	le <i>Query Profile</i> prouve la décorrélation : aucune boucle dans le plan

Et ensuite ?

Vous savez maintenant poser les questions négatives et comparatives, et aller chercher les N premiers de chaque groupe. Le LATERAL que vous venez d’écrire est le geste exact de l’as-of join du Cours 09 : au lieu de « les trois plus chers de cette sous-catégorie », vous demanderez « le dernier prix connu avant cette date ». La forme sera identique.

Avant d’y aller, deux étapes : le Cours 08 sur les données semi-structurées, où LATERAL sert à aplatir un tableau JSON, et les exercices de ce module, dans Exercices_corrigees_07.md.